EcoLoop

Circular material systems and zero-cost digital architecture.

Swami Sant Dass Public School, Jalandhar

The Linear Packaging Problem

Why conventional municipal recycling fails on modern multi-layer packaging.

The Material Reality

- **Multi-Layer Packaging (MLP):**
  Chips and snack packaging fuse plastic films with aluminum foil. Standard recyclers cannot separate these layers cost-effectively, diverting millions of tons directly to landfills.
- **Fragmented Logistics:**
  Communities and schools generate clean packaging waste, but lack direct channels connecting them to verified upcycling alternatives.
- **Information Asymmetry:**
  Consumers and institutions cannot verify whether their sorted recyclables are actually reprocessed or simply dumped.

The EcoLoop Approach

- **Mechanical Upcycling:**
  Heat-compressing clean multi-layer plastics into structural sheets that replace traditional plywood and particle boards.
- **Digital Transparency:**
  Publishing exact material specifications, density metrics, and inventory through an open platform.
- **Closed Institutional Loops:**
  Partnering directly with campuses to collect, process, and return finished utility goods back to the source.

The Physical Loop: Verified Materials

Six circular products systematically diverted from waste streams.

- **Eco-Bricks**
  High-density PET containers packed with clean flexible plastics.
- **MLP Sheets**
  Heat-pressed multi-layer packaging replacing standard plywood.
- **Bottle Planters**
  Upcycled PET containers for nursery and hydroponic gardening.
- **Uniform Bags**
  Heavyweight tote bags repurposed from discarded school uniforms.
- **Seed Paper**
  Zero-bleach handmade paper embedded with native plant seeds.
- **Bioenzymes**
  Citrus-fermented organic cleaning concentrates.



Reality Check: How It Started vs How It Is Going



[Meme: Me explaining to school teachers that collecting plastic wrappers requires a serverless cloud architecture]

Clown applying makeup / Roll Safe / SpongeBob presentation

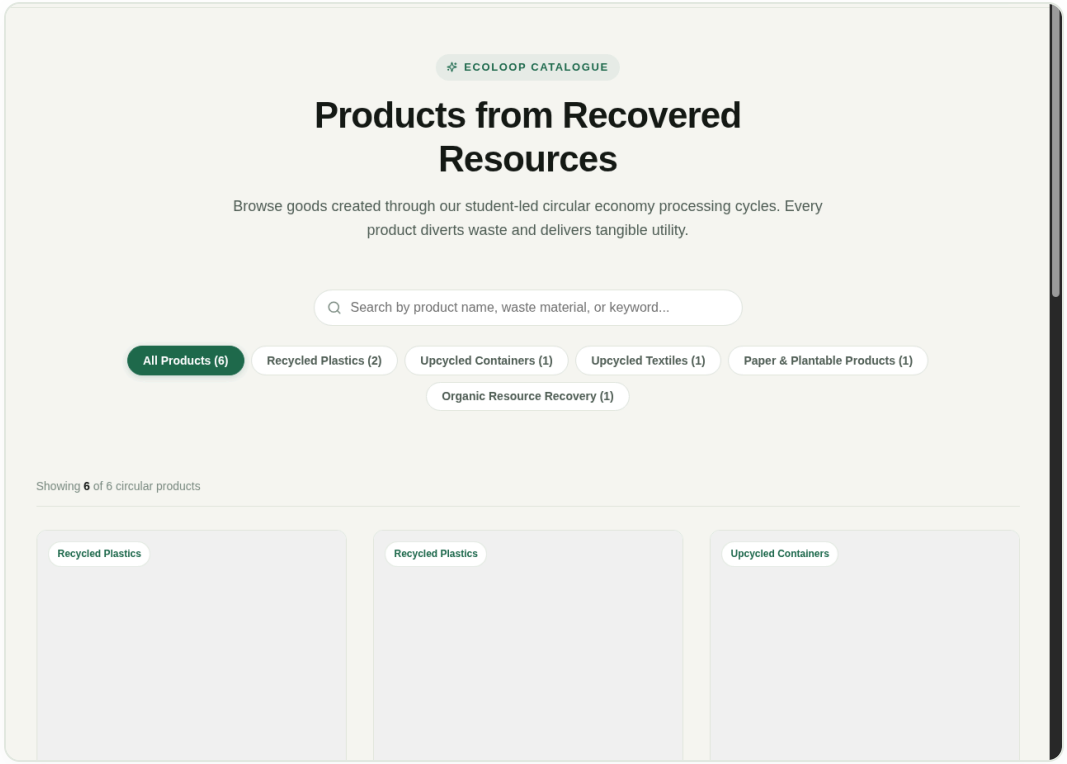
Started by collecting empty snack wrappers in school. Ended up building an automated cloud software platform.

Why Physical Projects Need Software

Moving from static school displays to an open, operational platform.

Platform Capabilities

- **Engineering Specifications:**
  Every product includes physical dimensions, density metrics, thermal characteristics, and production lead times.
- **Direct Transaction Routing:**
  Schools and community partners can query stock, request material samples, and submit institutional orders.
- **Modern Frontend Stack:**
  React 18 with TypeScript and Vite for fast client-side navigation and strict type safety.



The Zero-Cost Serverless Backend

Delivering full transactional workflows without servers or ongoing bills.

STEP 01

React Client

The user submits an inquiry; client-side validation ensures clean data before transmission.

→

STEP 02

Apps Script Webhook

A stateless Google Apps Script endpoint receives and parses the JSON payload securely.

→

STEP 03

Sheets Datastore

Appends structured records, updates inventory tracking, and triggers automated alerts.

Total Infrastructure Operating Cost

No credit card required. No servers to maintain. 99.9% uptime.

$0.00 / month

Reality Check: The Cloud Bill Dilemma



Buff Doge vs Cheems / Who would win

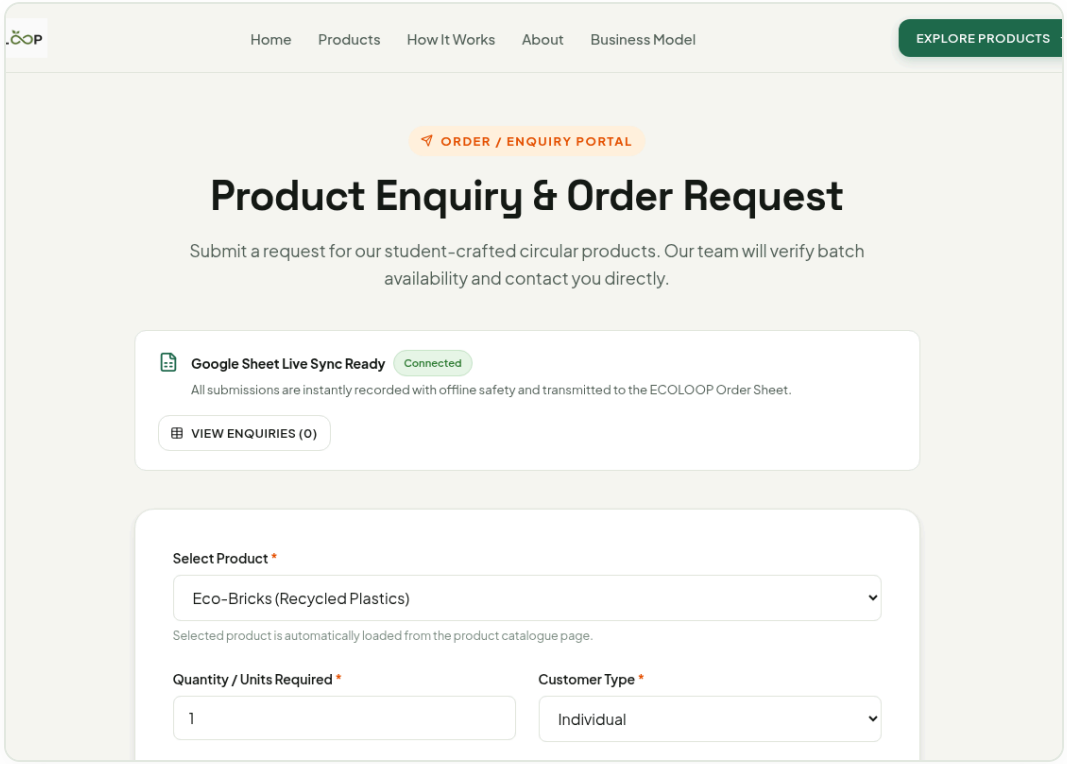
Enterprise developers arguing about Kubernetes microservices while two high schoolers run production on Google Sheets.

Interactive Inquiry Engine

Handling institutional orders and sample requests with context preservation.

Order & Sample Workflow

- **Contextual Pre-Filling:**
  Selecting an item automatically binds its product ID, specifications, and suggested order minimums into form state.
- **Client-Side Validation:**
  Immediate feedback flags incomplete or invalid entries before triggering network dispatches.
- **Confirmation & Logging:**
  A clean confirmation modal displays expected turnaround times while logging data directly to the spreadsheet.



The Paradigm Shift: Developer as Architect

How autonomous workflows shift focus from repetitive syntax to system design.

Traditional Coding

- Manually typing repetitive component boilerplate and state handlers.
- Spending hours searching forum threads to diagnose broken dependencies.
- Getting stuck on typos and syntax bugs instead of building user features.

Autonomous AI Coding

- You define the goals, system constraints, and visual design rules.
- The AI agent explores your files, writes components, and links routes.
- Automated tools test the code, surface errors, and repair bugs instantly.

Reality Check: Learning to Code in 2026



[Meme: Drake Hotline Bling - Reading 400 pages of framework documentation vs Writing clear specs and letting AI build]

Drake Hotline / Galaxy Brain

Reading 200 pages of manual documentation at 3:00 AM is officially a thing of the past. Define the requirements and let the tools execute.

How the Autonomous Coding Loop Works

Not a chatbot copying snippets, but an automated cycle that creates and tests code.

STAGE 01

Define the Goal

Write clear requirements, design rules, and data structures in a simple specification document.

STAGE 02

Autonomous Edit

The agent reads existing files, writes modular components, and links application routes.

STAGE 03

Automated Check

Automated tools build the code. If an error appears, the agent reads the terminal output and fixes it.

STAGE 04

Safe History

Every successful step is saved into version control, allowing you to experiment freely with zero risk.

Introducing OpenCode

An open-source autonomous coding agent that runs directly in your terminal.

Why OpenCode?

- **100% Open Source:**
  Maintained by Anomaly (GitHub: `anomalyco/opencode`). Free to use with zero vendor lock-in.
- **Dual Execution Modes:**
  `Plan Mode` surveys your project architecture without editing; `Build Mode` autonomously edits files and runs tests.
- **Connects to Any Model:**
  Works with free cloud endpoints or completely private offline models on your personal computer.

// Quick Installation in 60 Seconds

```
$ npm install -g @opencode/cli
$ cd ./my-project
$ opencode
```

Runs on macOS, Linux, and Windows WSL.

Free AI Models: Zero Cost for Students

Practical options you can use tonight without paying for subscriptions.

Free Cloud APIs

- **Google Gemini API:**
  Sign in to `aistudio.google.com` for a free key. Generous free tiers with Gemini 1.5 Flash and Gemini 2.0.
- **Groq Cloud:**
  Ultra-fast token generation on open-weight models like Llama 3.3 via `console.groq.com`.
- **GitHub Models:**
  Free student access to models like GPT-4o and Claude inside VS Code and GitHub Codespaces.

100% Offline with Ollama

- **Completely Private & Local:**
  Install from `ollama.com`. Runs entirely on your laptop hardware without needing an internet connection.
- **Recommended Model:**
  `qwen2.5-coder:7b` delivers outstanding coding logic on standard laptops with 8GB to 16GB of RAM.
- **Zero Latency:**
  Instant responses directly on device with zero dependence on external network connectivity.

Reality Check: The Compiler Never Lies



"I fixed all the bugs," says the AI model.
The compiler: 47 errors found in 3 files.

Thank You.

Scan the code to access the complete open-source repository, live demo, and student setup guide.



`github.com/Harman209/ecoloop`

Swami Sant Dass Public School, Jalandhar